

KCPerformance

Afbeeldingsoptimalisatie Projectenpagina's

Optimalisatie van afbeeldingen op de KCPerformance
Projectenpagina's voor een snellere website.

Tobias van de Ven
24-12-2024

Inhoud

Inhoud	1
Inleiding	2
Probleemanalyse	3
Analyse van de huidige situatie	4
Opslag structuur	4
Huidige bestandsgroottes	4
Geen rescaling	4
Conclusie	5
Oplossingen en toepassing.....	6
Aanpassen van bestaande bestanden	6
Opties	6
Beste optie	7
Implementatie	7
Beheer van nieuwe uploads.....	10
Opties	10
Beste optie	11
Implementatie	11
Resultatenanalyse	13
Bestandsgroottes	13
Laadtijden.....	13
Overige resultaten	13
Conclusie.....	14
Bijlagen	15

Inleiding

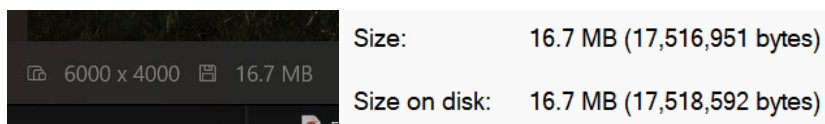
De KCPerformance website bevat een projectenpagina waar verschillende auto-projecten worden getoond. Deze projecten bevatten foto's van de voertuigen, die belangrijk zijn voor het presenteren van werk dat KCPerformance heeft geleverd. Echter, op dit moment laden de afbeeldingen op deze pagina veel te langzaam. Dit heeft niet alleen invloed op de gebruikerservaring, maar kan ook schadelijk zijn voor de algehele prestaties van de website, zoals laadtijden en SEO-ranking. Snelle laadtijden zijn van groot belang voor een professionele uitstraling en om gebruikerservaring te verbeteren. In dit document zal ik de huidige situatie analyseren en de oorzaken van de trage laadtijden onderzoeken. Vervolgens zal ik een oplossing implementeren met als doel de prestaties van de projectenpagina te verbeteren. Nadat de oplossing is uitgevoerd, zal ik de resultaten evalueren om te bepalen of de prestaties daadwerkelijk zijn verbeterd. Tot slot zal ik een conclusie trekken en beoordelen of de aangebrachte wijzigingen het gewenste effect hebben gehad.

Probleemanalyse

Tijdens mijn onderzoek naar de slome laadtijden van de projectenpagina, heb ik ontdekt dat de oorzaak ligt in de grootte van de afbeeldingen die op deze pagina's worden weergegeven. De afbeeldingen op de projectenpagina zijn namelijk in 4K-resolutie geüpload, wat veel te groot is voor gebruik op het web. Het gebruiken van Full HD (1920x1080) foto's meer dan voldoende om afbeeldingen scherp weer te geven zonder onnodige bestandsgroottes.

Door het gebruik van 4K-afbeeldingen is de bestandsgrootte van de foto's een stuk groter dan nodig, wat resulteert in langere laadtijden. Dit heb ik geanalyseerd via de netwerktool in Google Chrome, waarbij ik duidelijk kon zien dat de afbeeldingen traag laden door de grote bestandsgrootte. In de onderstaande screenshot is te zien dat de afbeeldingen langzaam laden en dat de bestandsgroottes veel groter zijn dan nodig.

	_DSC1585.jpg	200	jpeg	jquery.min.js:2	2.7 MB	2.50 s
	DSC07717.jpg	200	jpeg	jquery.min.js:2	2.5 MB	2.38 s
	DSC09210.jpg	200	jpeg	jquery.min.js:2	2.0 MB	1.98 s
	DSC1646.jpg	200	jpeg	jquery.min.js:2	17.5 MB	6.45 s



Afbeelding grote en bestandsgrote van afbeelding [DSC1646.jpg](#).

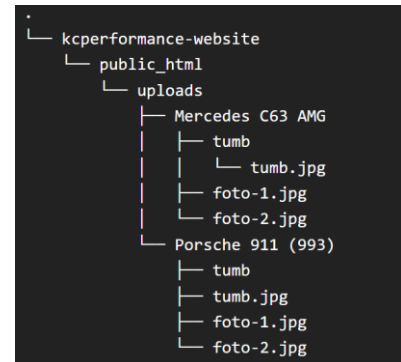
In het artikel "Need for mobile speed" wordt uitgelegd dat volgens onderzoek van Google de kans dat gebruikers een website verlaten met 53% toeneemt wanneer de laadtijd van een pagina langer is dan 3 seconden. Dit percentage zou zelfs nog hoger kunnen zijn als de laadtijd langer dan 5 seconden duurt. Dit laat zien hoe belangrijk het is om de laadtijd snel te houden voor een betere gebruikerservaring en minder bezoekers die de site snel verlaten.

Afbeeldingen spelen hierbij een grote rol, zoals Ralf van Veen zegt: "Afbeeldingen zijn vaak de grootste bestanden op een pagina en vertragen de laadtijd als ze niet geoptimaliseerd zijn." Door afbeeldingen goed te optimaliseren, kan de laadtijd sneller worden, wat de gebruikerservaring ten goede komt.

Analyse van de huidige situatie

Opslag structuur

De afbeeldingen die geüpload worden via het formulier, worden direct opgeslagen in een **uploads-folder** binnen de website-directory. Deze structuur bevat momenteel een groot aantal foto's, voornamelijk in 4K-resolutie. Er wordt dan een folder aangemaakt met de naam van de auto die is opgegeven in de form, en daar worden alle foto's in geplaatst. Ook wordt er een Tumb folder aangemaakt waar de hoofdfoto (Thumbnail) van het project in komt te staan.



Huidige bestandsgroottes

Omdat afbeeldingen in 4K worden geüpload zonder enige vorm van compressie of rescaling, nemen ze veel opslagruimte in beslag. De uploads folder is op het moment 3.5 GB groot dit is erg veel voor een folder op de server, als de folder nog meer wordt aangevuld kan dit zeker een probleem veroorzaken.

Size:	2.66 GB (2,860,921,560 bytes)
Size on disk:	2.66 GB (2,862,870,528 bytes)
Contains:	922 Files, 132 Folders

Voorbeeld: Een enkel 4K-afbeelding kan afhankelijk van de kwaliteit en compressie 4 MB tot 20 MB groot zijn. Voor een klein aantal aan foto's is dit nog wel te doen voor een webserver maar wanneer dit oploopt van honderd tot duizend- tallen zal dat zeker wel invloed hebben op de webserver.

Geen rescaling

Wanneer een afbeelding wordt geüpload, wordt deze eerst opgeslagen in de cache en vervolgens direct in de uploads-map geplaatst. Er is een poging gedaan om een rescaling-functie toe te voegen om de bestandsgrootte te verkleinen, maar deze functie werkt niet. Hierdoor blijven afbeeldingen in hun originele grootte opgeslagen, wat leidt tot de onnodig grote bestanden en trage laadtijden.

```

122 // Function to compress an image
123 function compressAndUploadImage($source, $destination, $quality) {
124     list($width, $height) = getimagesize($source);
125     $image = imagecreatefromjpeg($source);
126     $tmp_image = imagecreatetruecolor($width, $height);
127     imagecopyresampled($tmp_image, $image, 0, 0, 0, 0, $width, $height, $width, $height);
128     imagejpeg($tmp_image, $destination, $quality);
129     imagedestroy($image);
130     imagedestroy($tmp_image);
131 }
132 ?>
```

Conclusie

Uit de analyse blijkt dat er twee belangrijke problemen zijn die opgelost moeten worden:

1. **Bestaande bestanden**

De uploads folder bevat momenteel een groot aantal afbeeldingen in hoge resolutie (4K). Deze bestanden nemen te veel opslagruimte in beslag en zorgen voor lange laadtijden op de website. Om dit op te lossen, moeten de bestaande afbeeldingen worden gerescaled naar Full HD (1920x1080).

2. **Nieuwe bestanden**

Om te voorkomen dat dit probleem opnieuw ontstaat, moet ervoor gezorgd worden dat de afbeeldingen automatisch worden gerescaled voordat ze op de server worden geüpload. Dit zorgt ervoor dat er zonder enige moeite het probleem gelijkt wordt opgelost.

Oplossingen en toepassing

Er zijn dus 2 oplossingen nodig voor de bestanden die al bestaan en voor de bestanden die geüpload gaan worden zodat het probleem zich niet herhaalt.

Aanpassen van bestaande bestanden

Het doel is om de afbeeldingen zo efficiënt mogelijk te rescalen naar Full HD, terwijl de huidige structuur van de uploads-map behouden blijft. Dit zorgt ervoor dat de totale grootte van de uploads-folder verminderd wordt, en de foto's van 4K naar Full HD worden gescaled zonder verlies van kwaliteit.

Opties

Om het doel te behalen zijn er verschillende mogelijkheden ik heb ze uitgezocht:

1. Custom Python-script

Een custom Python-script gebruiken met bibliotheken zoals Pillow en OS, om door de folder structuur heen te gaan.

Voordelen:

- Zeer flexibel en efficiënt voor het verwerken van grote aantallen afbeeldingen.
- Het script kan eenvoudig worden aangepast om te werken met de huidige mappenstructuur.

Nadelen:

- Kennis van Python nodig voor het opzetten van het Script.
- Moet uitgevoerd worden op een server of lokaal systeem.

2. Rescale tool

Het gebruik van een specifieke tool of software, zoals ImageMagick of een grafische batchverwerkingsapplicatie, die afbeeldingen in bulk kan rescalen naar Full HD.

Voordelen:

- Snel en makkelijk in te stellen voor bulkverwerking van afbeeldingen.
- Makkelijk te gebruiken zonder technische kennis van programmeren.

Nadelen:

- Minder flexibel dan een custom Python-script; moeilijker om het proces volledig te automatiseren met specifieke eisen.
- Handmatige afbeeldingen vervangen voor de originele bestanden.

3. Handmatige batchverwerking

Het handmatig verwerken van de afbeeldingen met software zoals Adobe Photoshop of GIMP, waarbij de bestaande één voor één worden gerescaled.

Voordelen:

- Gemakkelijk voor kleine hoeveelheden afbeeldingen of als het maar af en toe moet.
- Eenvoudig in te stellen zonder extra tools of code.

Nadelen:

- Tijdsintensief en niet schaalbaar voor een grote hoeveelheid afbeeldingen.
- Niet geautomatiseerd, wat betekent dat de werklast hoog is bij veel bestanden.

Beste optie

Ik denk dat het makkelijkste is om een Python-script te gebruiken om de afbeeldingen te rescalen. Zo kan ik makkelijk door de mappen navigeren, de afbeeldingen scalen en dezelfde namen behouden. Daarna kunnen ze zonder problemen weer op de website worden geüpload.

Implementatie

Gebruik bij dit deel van de documentatie de python code voor automatische afbeeldingresizing in mappen.

Overzicht

Deze Python code is een handige tool voor het beheren en optimaliseren van afbeeldingen in mappen. Het programma gebruikt de **Pillow**-library om afbeeldingen te verkleinen naar een maximale resolutie van **1920x1080 pixels**, terwijl de originele verhoudingen behouden blijven.

Het script doorloopt een mappenstructuur, controleert afbeeldingen in alle sub mappen en past alleen wijzigingen toe op bestanden met de extensies `.jpg`, `.jpeg` of `.png`. Dit maakt het ideaal voor het opschonen van grote afbeelding collecties of het voorbereiden van bestanden voor het web.

Benodigde libraries

- **Os:** Voor het doorlopen van mappen en bestanden.
- **Pillow (PIL):** Voor het aanpassen van afbeeldingen, zoals schalen en opslaan.

Hoe werkt het

1. De gebruiker voert de locatie in van de map met afbeeldingen.
2. Het script doorzoekt de hele map en alle sub mappen.

3. Afbeeldingen worden verkleind naar de maximale resolutie zonder verlies van verhoudingen.
4. Mappen met “thumb” in de naam worden apart verwerkt.

Code uitleg

Afbeeldingen verkleinen

De `resize_image()` functie opent een afbeelding met de Pillow-library, schaaft deze naar 1920x1080, en slaat de afbeelding op dezelfde locatie op:

```
def resize_image(image_path):  
    with Image.open(image_path) as img:  
        img.thumbnail((1920, 1080)) # Schaalt naar maximaal 1920x1080 pixels  
        img.save(image_path)        # Overschrijft het originele bestand
```

Waarom `thumbnail()`? Dit zorgt ervoor dat de aspect ratio (verhouding tussen breedte en hoogte) behouden blijft, zodat afbeeldingen niet vervormd worden.

Mappen verwerken

De `process_folder()` functie doorloopt de bestanden in een specifieke map. Alleen bestanden met een afbeelding-extensie zoals `.jpg` of `.png` worden verwerkt:

```
def process_folder(folder_path):  
    for filename in os.listdir(folder_path):  
        file_path = os.path.join(folder_path, filename)  
        if os.path.isdir(file_path):  
            continue # Sla submappen over  
        if filename.lower().endswith(('jpg', 'jpeg', 'png')):  
            print(f'Resizing image {file_path}')  
            resize_image(file_path)
```

Controle op bestand-extensies, alleen `.jpg`, `.jpeg` en `.png` bestanden worden herkend, wat voorkomt dat het script andere bestandstypen probeert te openen.

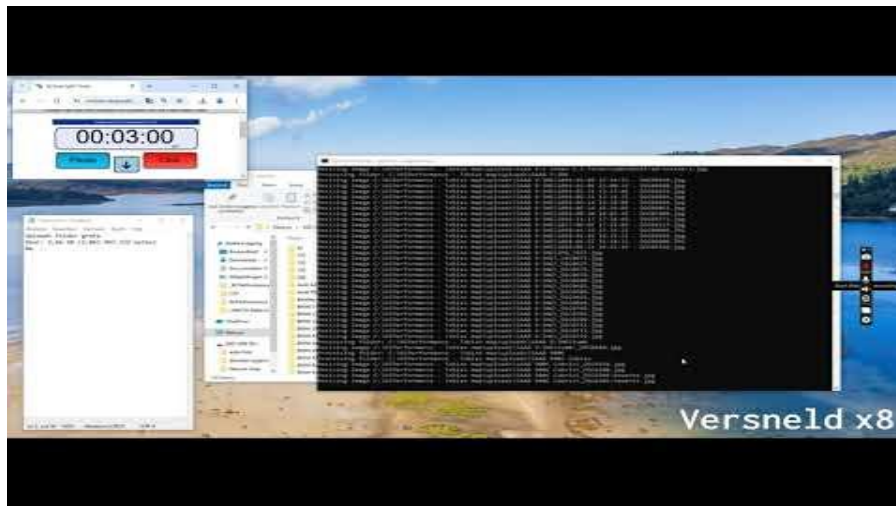
Map structuur doorlopen

Met de `os.walk()` functie wordt de hele mappen structuur doorzocht. Sub mappen worden automatisch geïdentificeerd en afzonderlijk verwerkt:

```
for dirpath, dirnames, filenames in os.walk(root_directory):  
    if 'thumb' in dirpath.lower():  
        print(f'Processing thumb folder: {dirpath}')  
        process_folder(dirpath)  
    elif dirpath != root_directory:  
        print(f'Processing folder: {dirpath}')  
        process_folder(dirpath)
```

Mappen met “thumb” krijgen speciale aandacht dit is handig als je die afbeeldingen anders zou willen beheren.

Demonstratie



Bekijk de video: Showcase – Image Scale. In deze video wordt gedemonstreerd hoe de tool in praktijk wordt toegepast om het gestelde probleem op te lossen.

De video demonstreert

1. Het starten van de code

De code wordt uitgevoerd in een terminal of IDE. Je wordt gevraagd om de locatie van de map op te geven.

2. Uploads folder

De uploads folder van de afbeeldingen voor projecten is gedownload van de server. Ik toon de afbeelding groottes en de totale folder grote.

3. Proces

In de video kun je in de terminal zien hoe het script alle afbeeldingen doorloopt en schaaft.

4. Resultaat

Ik laat zien dat de afbeeldingen kleiner zijn geworden in resolutie en de totale grote van de uploads folder aanzienlijk kleiner is.

Beheer van nieuwe uploads

Het doel is om te voorkomen dat toekomstige uploads dezelfde problemen veroorzaken door afbeeldingen automatisch te rescalen naar Full HD tijdens het uploadproces. Hierdoor blijft de opslag efficiënt en worden bestanden uniform verwerkt zonder extra handmatige stappen.

Opties

Om het doel te behalen zijn er verschillende mogelijkheden ik heb ze uitgezocht:

1. Gebruiker laat afbeeldingen rescalen

De verantwoordelijkheid ligt bij de gebruiker om afbeeldingen zelf te rescalen voordat ze geüpload worden.

Voordelen:

- Geen aanpassingen nodig in de code.
- Werkt goed als de gebruikers technisch ervaren zijn.

Nadelen:

- Gebruikers maken mogelijk fouten of vergeten deze stap.
- Extra handmatige actie kan tijd kosten.

2. Automatisch rescalen met PHP/JavaScript

Een stukje PHP of JavaScript wordt toegevoegd aan het uploadproces om afbeeldingen automatisch te rescalen naar Full HD.

Voordelen:

- Volledig geautomatiseerd en voorkomt menselijke fouten.
- Kan direct geïntegreerd worden in het huidige uploadproces.

Nadelen:

- Vereist aanpassing van bestaande code.
- Klein prestatieverlies tijdens het uploaden door het rescalen.

3. Server-side processing na upload

Afbeeldingen worden in originele kwaliteit geüpload en vervolgens op de server geresized naar Full HD.

Voordelen:

- Geen extra belasting voor de gebruiker.
- Kan complexere verwerkingen doen, zoals watermerken of formaten aanpassen.

Nadelen:

- Meer opslagruimte vereist voor originele uploads.
- Extra serverresources nodig voor het verwerken van grote aantallen bestanden.

Beste optie

De beste keuze is optie 2: **Automatisch rescalen met PHP/JavaScript**. Door dit proces vooraf aan de upload te automatiseren, blijft het systeem gebruiksvriendelijk zonder extra handmatige stappen. Dit voorkomt fouten en zorgt ervoor dat afbeeldingen altijd voldoen aan de gewenste specificaties voordat ze worden opgeslagen.

Implementatie

Gebruik bij dit deel van de documentatie het PHP bestand, geoptimaliseerde compress and upload image functie.

Overzicht

Ik heb deze functie aangepast en werkend gemaakt om ervoor te zorgen dat afbeeldingen direct tijdens het uploadproces worden geschaald naar een maximale resolutie van 1920x1080 pixels. Hierdoor worden afbeeldingen automatisch geoptimaliseerd voordat ze op de server terecht komen.

Functionaliteiten

1. **Afbeeldingen schalen en aspect ratio behouden**

De functie past de grootte van de afbeeldingen aan tot maximaal 1920x1080 pixels, waarbij de verhoudingen behouden blijven om vervorming te voorkomen.

2. **Compressie toepassen**

Door een kwaliteitswaarde mee te geven, wordt de bestandsgrootte van de afbeelding verminderd zonder merkbaar kwaliteitsverlies.

3. **Geoptimaliseerde opslag**

De aangepaste afbeelding wordt direct opgeslagen op de server, wat opslagruimte bespaart en de laadtijd verbetert.

Code uitleg

Functieparameters

De functie `compressAndUploadImage()` accepteert drie parameters:

- **\$source:** Het pad naar de originele afbeelding.
- **\$destination:** Het pad waar de geoptimaliseerde afbeelding wordt opgeslagen.
- **\$quality:** De compressiekwaliteit (0-100, waarbij de 100 hoogste kwaliteit is).

Afmetingen controleren en aanpassen

De functie berekent of de afbeelding groter is dan de maximale afmetingen (1920x1080). Als dat zo is, worden de nieuwe afmetingen berekend met behoud van de verhoudingen.

```
list($width, $height) = getimagesize($source);

$max_width = 1920;
$max_height = 1080;

if ($width > $max_width || $height > $max_height) {
    $aspect_ratio = $width / $height;
    if ($aspect_ratio > 1) {
        // Landscape image
        $width = $max_width;
        $height = round($width / $aspect_ratio);
    } else {
        // Portrait image
        $height = $max_height;
        $width = round($height * $aspect_ratio);
    }
}
```

Afbeelding resizen en opslaan

Met behulp van `imagecreatefromjpeg()` wordt de afbeelding ingeladen, waarna `imagecreatetruecolor()` een nieuwe afbeelding met de juiste afmeting aanmaakt. Vervolgens wordt de originele afbeelding geschaald en opgeslagen.

```
$image = imagecreatefromjpeg($source);
$tmp_image = imagecreatetruecolor($width, $height);
imagecopyresampled($tmp_image, $image, 0, 0, 0, 0, $width, $height, getimagesize($source)[0], getimagesize($source)[1]);
imagejpeg($tmp_image, $destination, $quality);
imagedestroy($image);
imagedestroy($tmp_image);
```

Voordelen van deze aanpak

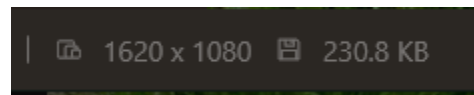
- **Directe optimalisatie:** Afbeeldingen worden direct bij uploaden verwerkt, waardoor extra verwerking achteraf niet nodig is.
- **Snellere laadtijden:** Kleinere bestanden laden sneller op websites, wat resulteert in betere prestaties.
- **Serverbeheer:** Minder opslagruimte wordt gebruikt, wat kosten bespaart en de server minder belast

Resultatenanalyse

Na de implementatie van de oplossingen heb ik de resultaten geanalyseerd door verschillende meetpunten te vergelijken: bestandsgroottes en laadtijden met Chrome dev tools.

Bestandsgroottes

De bestandsgroottes van afbeeldingen zijn aanzienlijk afgenomen. Eerst waren de bestanden in 4K-resolutie gemiddeld tussen de 4 MB en 20 MB per afbeelding. Na de optimalisatie (rescaleing naar 1920x1080 pixels en compressie) is de gemiddelde bestandsgrootte verkleind naar ongeveer 500 KB tot 1 MB per afbeelding. Dit betekent een reductie van meer dan 80%, afhankelijk van het originele bestand.



Laadtijden

Door de verminderde bestandsgrootte zijn de laadtijden van de projectenpagina's sterk verbeterd. Waar het eerder 5 tot 7 seconde duurde om alle afbeeldingen te laden op de pagina, is de laadtijd nu minder dan 2 seconden.

Neem als voorbeeld een afbeelding [DSC00174.jpg](#) op de KCPerformance website voorheen 6.45 seconden, nu bijna 50% verlaagd naar 3.54 seconden.

DSC00172.jpg	200	jpeg	bentley-continental-	1.2 MB	3.66 s
20240416_120358.jpg	200	jpeg	bentley-continental-	1.3 MB	3.79 s
DSC00171.jpg	200	jpeg	bentley-continental-	1.3 MB	3.54 s
DSC00174.jpg	200	jpeg	bentley-continental-	1.4 MB	3.54 s

Overige resultaten

- **Opslagbesparing:** De totale opslagruimte van de “uploads” folder is gedaald van 2 GB naar minder dan 800 MB. Dit voorkomt toekomstige serverproblemen en maakt de website schaalbaar.
- **Consistentie:** Nieuwe uploads worden automatisch geschaald en gecomprimeerd, wat een consistente kwaliteit en bestandsgrootte garandeert zonder extra handmatige stappen

Conclusie

De optimalisatie van de afbeeldingen op de projectenpagina heeft duidelijk gezorgd voor een betere werking van de KCPerformance-website. Door de afbeeldingen automatisch te verkleinen naar Full HD (1920x1080) en te comprimeren, zijn de belangrijkste problemen met laadtijden en opslag opgelost.

Wat is verbeterd:

- De bestandsgroottes van afbeeldingen zijn flink verkleind, van gemiddeld 4-20 MB naar 500 KB-1 MB. Dit zorgt voor meer dan 80% ruimtebesparing.
- De laadtijd van de projectenpagina is verkort van 5-7 seconden naar minder dan 2 seconden, wat een betere gebruikerservaring oplevert.

Andere voordelen:

- De opslagmap voor afbeeldingen is nu veel kleiner, wat problemen met beperkte ruimte in de toekomst voorkomt.
- Dankzij de automatische optimalisatie zijn nieuwe uploads direct in het juiste formaat, wat het werkproces eenvoudiger maakt.

De oplossing heeft zowel de snelheid als de gebruikerservaring sterk verbeterd. Om deze resultaten vast te houden, is het aan te raden om regelmatig de prestaties te blijven controleren en eventuele toekomstige verbetering aan te pakken.

Bijlagen

Relevante bestanden die nodig zijn bij deze documentatie

1. **Python image scale code**
imageScale.py
2. **Documentatie Python code**
Documentatie_Python_image_rescale_markdown.pdf
3. **Python showcase video (als die niet werkt)**
Showcase – Image Scale.mp4
<https://youtu.be/bkwdHuvxWTI>